

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: Генетические алгоритмы**

Студент гр. 4382

\_\_\_\_\_

Пермитин В. А.

Руководитель

\_\_\_\_\_

Жангиров Т. Р.

Санкт-Петербург

2026

**ЗАДАНИЕ  
НА УЧЕБНУЮ ПРАКТИКУ**

Студент: Пермитин В. А.

Группа: 4382

Тема практики: Генетические алгоритмы

Задание на практику:

Для заданного неориентированного графа  $G = (V, E)$ , необходимо найти множество вершин  $S$ , которое является вершинным покрытием графа и минимально.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 00.00.0000

Дата защиты отчета: 00.00.0000

|              |       |                |
|--------------|-------|----------------|
| Студент      | _____ | Пермитин В. А. |
| Руководитель | _____ | Жангиров Т. Р. |

## АННОТАЦИЯ

Цель работы: познакомиться с генетическими алгоритмами для решения задач оптимизации и поиска

Языки программирования: Python3, C++, C#, Java, Golang, Javascript

Общее описание заданий: поиск аппроксимирующей функции, приближение функций, нахождение экстремумов, решение оптимизационных задач с использованием генетических алгоритмов (методичка будет выложена)

Бригады: от 1 до 3 человек. Две бригады могут взять одно и то же задание, если реализация будет на разных ЯП.

Требование к решению:

Программа должна иметь GUI.

Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.

Алгоритмы реализуются самостоятельно без использования готовых решений.

Настройкой параметров алгоритмов должна производиться пользователем.

Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключение между разными текущими решениями.

Возможность перейти к конечному решению пропустив отображение всех шагов.

Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

Доп. баллы даются за:

Реализацию возможности вернуться на несколько шагов назад.

Этапы выполнения задания:

- 1) Формирование бригады, выбор задания и ЯП, распределение ролей в бригаде
- 2) Демонстрация прототипа GUI и плана по решению задачи (описание формата данных, используемой функции качества, и.т.д.)
- 3) Промежуточная сдача программы (несколько итераций). На данном этапе должен быть частично-функционирующий GUI, работающий алгоритм, но возможно, требующий корректировок.
- 4) Демонстрация финальной версии программы и отчета.

## СОДЕРЖАНИЕ

|                                                           |    |
|-----------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                            | 6  |
| 1 ПОСТАНОВКА ЗАДАЧИ .....                                 | 7  |
| 1.1 Предметная область .....                              | 7  |
| 1.2 Описание формата данных.....                          | 7  |
| 1.3 Задачи практики .....                                 | 7  |
| 1.4 Решение задач.....                                    | 8  |
| 1.4.1 Виды кодирования .....                              | 8  |
| 1.4.2 Подбор целевой функции генетического алгоритма..... | 8  |
| 1.4.3 Выбор метода скрещивания .....                      | 9  |
| 1.4.4 Выбор метода отбора .....                           | 9  |
| 1.4.5 Выбор метода мутации .....                          | 10 |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ .....                                 | 11 |
| 2.1. Формулировка задачи 1.....                           | 11 |
| 2.2. Формулировка задачи 2.....                           | 11 |
| 2.3. Формулировка задачи 3.....                           | 11 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....                           | 12 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ.....                                 | 13 |
| ЗАКЛЮЧЕНИЕ .....                                          | 14 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                    | 15 |
| ПРИЛОЖЕНИЕ А .....                                        | 16 |

## **ВВЕДЕНИЕ**

Вершинное покрытие – это такое подмножество вершин графа  $G(V, E)$ , что для любого ребра существует такая вершина из подмножества вершин, что эта вершина является концом ребра.

Задача минимального вершинного покрытия – это задача поиска минимального по включению вершинного покрытия.

Для решения этой задачи будет реализован генетический алгоритм.

# 1 ПОСТАНОВКА ЗАДАЧИ

## 1.1 Предметная область

Генетические алгоритмы активно применяются в обучении с подкреплением, помогают подбирать параметры для моделей машинного обучения, помогают решать задачи без математического представления (например, поиск подходящих цветов)

## 1.2 Описание формата данных

Был реализован генетический алгоритм в классе GeneticAlgorithm. Его конструктор принимает на вход:

- 1) Входную матрицу смежности (причем необязательно симметричную, симметричность проверяется в GUI): List[List[float]]
- 2) Размер популяции: int
- 3) Штраф за нарушение жесткого ограничения: int
- 4) Вероятность скрещивания родителей: float
- 5) Вероятность мутации одного из родителей: float
- 6) Размер турнира: int
- 7) Количество элиты: int

Для запуска генетического алгоритма был реализован метод run. Он принимает на вход только один int – это количество итераций генетического алгоритма. На выходе метод выдает самую приспособленную особь и массив максимальной приспособленности на каждой итерации.

## 1.3 Задачи практики

Задачи:

- 1) Выбор метода кодирования индивидуума
- 2) Подбор целевой функции для генетического алгоритма
- 3) Выбор метода скрещивания
- 4) Выбор метода отбора
- 5) Выбор метода мутации
- 6) Реализация генетического алгоритма
- 7) Реализация графического интерфейса

## 1.4 Решение задач

### 1.4.1 Виды кодирования

Мной были рассмотрены следующие виды кодирования:

- 1) Бинарная кодировка - решение представлено в виде массива 0 и 1
- 2) Порядковая кодировка - решение представлено в виде массива натуральных чисел, где учитывается их порядок
- 3) Вещественная кодировка - решение представлено в виде массива вещественных чисел. Целые числа частный случай.

Моя задача – найти наименьшее вершинное покрытие. То есть, нужно найти такое подмножество вершин, что хотя-бы один конец каждого ребра входит в это подмножество. Я решил представить подмножества вершин с помощью бинарной кодировки, где если ген равен 1, то вершина включена, если ген равен 0, то ген не в подмножестве.

Порядок выбора вершин не важен, а решение не нужно представлять в виде вещественных чисел, значит, мой выбор – бинарная кодировка.

### 1.4.2 Подбор целевой функции генетического алгоритма

Генетический алгоритм ищет самых приспособленных индивидуумов, так что мне нужно максимизировать функцию приспособленности для тех индивидуумов, у которых вершинное покрытие имеет наименьшее количество вершин. Поэтому введу два ограничения:

- 1) Жесткое ограничение – не должно быть ребер, оба конца которых не находятся в подмножестве
- 2) Еще одно ограничение – вершинное покрытие должно быть минимальным

Для выполнения этих ограничений были реализованы штрафы.

Пусть  $k$  – количество ребер, которые не покрываются текущим решением,  $p$  – количество вершин. А  $n$  – количество вершин в покрытии. Тогда целевая функция будет следующая:  $p - n - k * \text{ШТРАФ}$ , где ШТРАФ ( $> 1$ ) – размер штрафа за непокрытое ребро. ШТРАФ вводится



для того, чтобы решение с наибольшим количеством непокрытых ребер точно отверглось.

### **1.4.3 Выбор метода скрещивания**

Для скрещивания двух родителей я реализовал равномерное скрещивания.

Я ознакомился со следующими видами скрещиваний:

- 1) Одноточечное скрещивание
- 2) Двухточечное скрещивание
- 3) Равномерное скрещивание
- 4) Упорядоченное скрещивание
- 5) Скрещивание смещением
- 6) Имитация двоичного скрещивания

Было выбрано равномерное скрещивание, потому что если в графе есть  $n$  вершин, то существует  $n!$  способов переставить порядок этих вершин, при этом все эти графы будут изоморфны друг другу. Можно расставить вершины так, чтобы одноточечное или двухточечное скрещивание выбирало в основном выбирало такие точки скрещивания, что родители будут обмениваться минимальным количеством вершин из минимального вершинного покрытия. Значит, такой метод в таком случае будет сходиться не так быстро.

Я выбрал равномерное скрещивание, потому что такое скрещивание дает наибольшее разнообразие подмножеств вершин, что мне и нужно.

### **1.4.4 Выбор метода отбора**

Я ознакомился со следующими видами отбора:

- 1) Правило рулетки
- 2) Стохастическая универсальная выборка
- 3) Ранжированный отбор
- 4) Масштабирование приспособленности
- 5) Турнирный отбор
- 6) Отбор усечением

7) Элитарный отбор

8) Отбор вытеснением

Мной был реализован элитарный отбор с добором через турнирный отбор.

Элитарный отбор лучше всего подходит в этой задаче, потому что если было найдено лучшее решение, то при его мутировании и скрещивании всегда можно найти решение лучше, если оно есть и алгоритм не застрял на нескольких почти неизменяющихся индивидуумах (преждевременная сходимость).

Турнирный отбор был выбран, потому что он помогает оставлять лучшие решения в популяции, и при этом дает разнообразие, которое помогает избежать преждевременной сходимости.

#### **1.4.5 Выбор метода мутации**

Я ознакомился со следующими видами мутации:

1) Инвертирование бита

2) Мутация обменом

3) Мутация обращением

4) Мутация перетасовкой

5) Вещественная мутация

Я реализовал мутацию инвертированием бита, потому что данная мутация позволяет случайно включить или исключить вершину из множества, добавляя большее разнообразие, и тем самым перебирая возможные варианты решения.

## **2 РЕЗУЛЬТАТЫ РАБОТЫ**

Данная глава должна содержать описание процесса решения и результаты по каждой задаче практики из предыдущего раздела. По приведённому тексту читатель должен понять, *что* и *как* было сделано для выполнения поставленных задач.

Повествование ведется обезличенно.

### **2.1. Формулировка задачи 1**

Описание процесса решения задачи №1, реализованные или использованные классы / модули / сервисы и т.д.

### **2.2. Формулировка задачи 2**

...

### **2.3. Формулировка задачи 3**

...

### **3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ**

Для решения данной задачи были использованы следующие технологии:

- 1) Python
- 2) Tkinter
- 3) Matplotlib
- 4) Networkx

#### 4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша\_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

##### Отзыв

##### о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

## **ЗАКЛЮЧЕНИЕ**

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Исходный код программы для решения задачи поиска минимального вершинного покрытия [Электронный ресурс]. URL: [https://github.com/vapermitin/genetic\\_algorithms\\_practice](https://github.com/vapermitin/genetic_algorithms_practice) (дата обращения: 08.07.2026).

## **ПРИЛОЖЕНИЕ А**